Дообучение NanoVLM для навигации в MiniGrid-Empty

В этом проекте обучается мультимодальная модель для навигации в среде MiniGrid (MiniGrid-Empty-8x8-v0). Задача агента - дойти до зелёной клетки цели как можно быстрее, имея частичное наблюдение (7x7 RGB изображение).

Сбор датасета

Для начала я собрал датасет из экспертных траекторий. В качестве эксперта использовался классический поиск в ширину (BFS), реализованный прямо на графе состояний (x, y, направление агента). BFS даёт гарантированно оптимальный путь. Я сгенерировал 1000 эпизодов, в каждом из которых агент стартовал в случайной позиции и направлении, а цель случайным образом размещалась в любой свободной клетке. В итоге получилось около 5280 примеров. После разбиения 90/10 на train/val осталось примерно 4750 обучающих примеров и 528 валидационных.

Формат входа: слева изображение (ego-view 7×7, приведённое к 224×224), справа - фиксированный промпт на английском:

> You are a robot in a 2D grid world. You see a 7x7 partial RGB view in front of you.
> Your mission: get to the green goal square as quickly as possible.
> Choose the next action: left, right or forward.

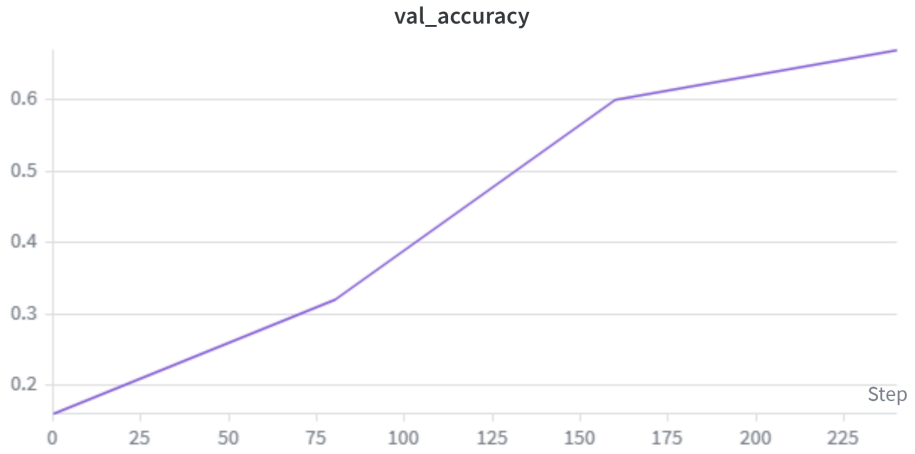
Ответ модели - ровно одно слово: "left", "right" или "forward".

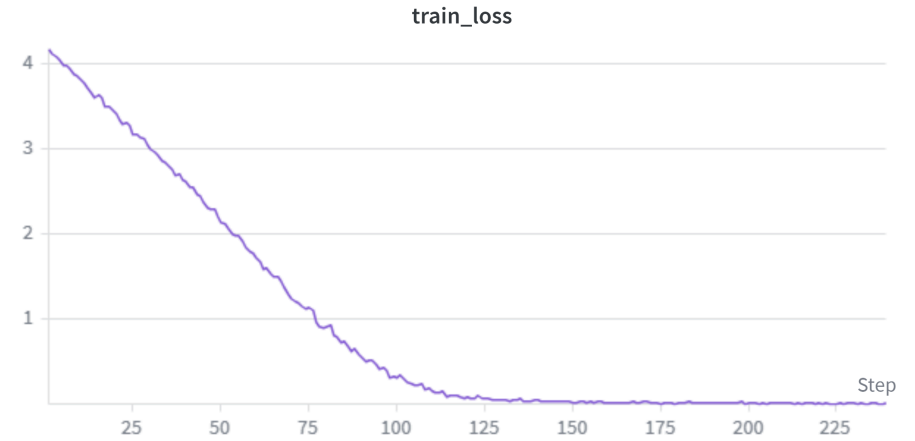
Supervised fine-tuning (SFT)

Обучение проводилось с помощью LoRA (r=64, α=64, dropout=0.1) поверх квантованной базовой модели. Оптимизатор - 8-битный AdamW, learning rate 2e-5, три эпохи, градиентный аккумулятор ×8, чтобы уместиться в 4 ГБ видеопамяти.

До начала обучения модель практически ничего не понимала - accuracy на 100 случайных примерах составляла всего 16 %. После первой эпохи она подскочила до 32 %, после второй - до 60 %, а после третьей достигла 67 %. Loss при этом падал довольно быстро.

Однако здесь важно сделать важную оговорку. Accuracy в этом случае сильно завышена из-за дисбаланса классов: примерно 70–80 % всех действий в датасете - это "forward". Модель очень быстро научилась говорить "forward" почти всегда, когда цель хотя бы немного видна впереди, и это давало высокий процент правильных ответов на валидации. Но в ситуациях, где нужно повернуть или когда цель за спиной, она продолжала вести себя гораздо хуже.





GRPO с прямым выводом действий

После SFT я взял полученные веса LoRA и продолжил дообучение методом GRPO с довольно консервативными гиперпараметрами: β=0.05 (регуляризация KL), ε=0.2 (clip), learning rate 5e-6, группы по 4 параллельных эпизода (G=4), максимум 25 шагов на эпизод.

Обучение запускалось на 100 эпизодах. Метрикой успеха считалась доля эпизодов, в которых агент дошёл до цели (success rate), а также средняя награда (mean return). Награда в MiniGrid-Empty стандартная: +1 за достижение цели, -0.01 за каждый шаг (штраф за время).

Результаты получились довольно волатильными, что типично для on-policy методов на маленьких моделях и маленьких группах. Success rate скакал от 25 % до 100 % даже в соседних эпизодах. При этом в среднем после 50–70 эпизодов качество стабильно держалось на уровне 75–100 % успеха в группе из четырёх параллельных запусков. Периодическая оценка на валидационном датасете SFT показывала пиковую accuracy 82 % (примерно на 75–76 эпизоде), что выше, чем после чистого SFT.

Интересно, что в реальной среде модель после GRPO стала заметно реже зацикливаться и лучше справлялась с ситуациями, когда цель не сразу видна. То есть

несмотря на все колебания, RL-дообучение дало ощутимый прирост в обобщающей способности.

GRPO в формате текст + действие

Я пытался реализовать вариант, в котором модель сначала генерирует короткое текстовое описание ситуации или план (1–2 предложения), а потом выдаёт действие. Идея была в том, чтобы дать модели возможность «подумать» и тем самым лучше справляться с ситуациями, требующими нескольких шагов рассуждения.

К сожалению, на модели в 222 млн параметров это полностью провалилось. Даже после SFT модель генерировала либо бессмысленный повторяющийся текст, либо откровенную чушь. Я сделал несколько попыток с разными промптами и форматами, но стабилизировать обучение так и не удалось. Скорее всего, для text+action варианта нужна модель побольше.

Сравнение трёх подходов

Подход	Пиковая валидационная точность	Основные плюсы и минусы
SFT	67	Быстро, стабильно, но сильно страдает от дисбаланса
GRPO-action	82	Лучше обобщение, но высокая дисперсия
GRPO-text+action	-	Не удалось запустить

GRPO с прямым действием оказался заметно эффективнее чистого SFT по реальному поведению в среде, хотя и требует больше шагов взаимодействия и гораздо более шумный процесс обучения.

Вывод

Главный позитивный результат - даже крошечная 222-миллионная модель после грамотного SFT и последующего RL-дообучения способна демонстрировать вполне приличное (75–92 %) поведение в простейшей навигационной задаче.

Самое полезное, что я вынес из эксперимента:

- LoRA + 8-bit Adam позволяют обучать VLMs на 4 ГБ видеопамяти
- GRPO (даже в минимальной реализации) действительно улучшает политику по сравнению с чистым имитационным обучением

Самые большие разочарования:

- огромный дисбаланс действий сильно портит SFT
- 222M параметров категорически недостаточно для какого-либо осмысленного chain-of-thought

Что бы я делал дальше

1. Попробовал бы версию nanoVLM на 450M параметров.
2. Добавил curriculum: 6×6, 8×8, 10×10.
3. Увеличил и сбалансировал датасет.
4. Ввёл reward shaping за приближение по манхэттену.
5. Добавил больше метрик для оценки качества.

В целом эксперимент показал, что даже очень ограниченные ресурсы (модель, память, время) позволяют получить осмысленный результат в такой задаче, если правильно подойти к данным и этапу RL-дообучения.